

COMS4045A – Assignment

Lisa Pitsi 2590083
Aluwani Nndwamato 2671832

May 16, 2026

1 Prerequisites

- Fully installed ROS Kinetic and TurtleBot3 Kinetic packages.
- Verify ROS version – `rosversion -d` (should return kinetic)
- Source environments – `source /opt/ros/kinetic/setup.bash, source ~/yumi_ws/devel/setup.bash`
- Unpack workspace – `tar -zxvf robot_assignment_ws.tar.gz`
- Compile workspace – `rm -rf robot_assignment_ws.tar.gz, cd robot_assignment_ws, rm -rf build devel install`
- Create package – `catkin_create_pkg surveillance_bot rospy std_msgs geometry_msgs sensor_msgs nav_msgs`
- Create launch folder – `mkdir -p surveillance_bot/launch`
- Create launch file – `nano surveillance_bot/launch/gmapping.launch` (paste XML, save and exit)
- Build workspace – `cd ~/robot_assignment_ws, catkin_make`
- Run – `source devel/setup (add to .bashrc), ./startWorld`

2 Mapping

- Install teleoperation – `sudo apt install ros-kinetic-teleop-twist-keyboard`
- Start simulation – `clear, ./startWorldkeyboard`
- Launch SLAM – `roslaunch surveillance_bot gmapping.launch`
- Start teleoperation – `roslaunch teleop_twist_keyboard teleop_twist_keyboard.py /cmd_vel:=/cmd_vel_mux/input/teleop`
- Control robot – Use keyboard keys to navigate and map the environment.

```
ros@ASUSZenbook:~$ roslaunch teleop_twist_keyboard teleop_twist_keyboard.py
/cmd_vel:=/cmd_vel_mux/input/teleop
Reading from the keyboard and Publishing to Twist!
-----
Moving around:
u    i    o
j    k    l
m    ,    .

For Holonomic mode (strafing), hold down the shift key:
U    I    O
J    K    L
M    <    >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:      speed 0.5      turn 1.0
```

- Save map – `roslaunch map_server map_saver -f my_map, mkdir -p ~/robot_assignment_ws/src/surveillance_bot/maps`
`cp my_map.yaml ../maps, cp my_map.pgm ../maps`

Creates:

- `my_map.pgm` → occupancy grid
- `my_map.yaml` → resolution, origin, thresholds

3 Planning

We used PRM (Probabilistic Roadmap) because the same graph can be reused for multiple navigation queries, making it more efficient than RRT. The `my_map.pgm` file is converted into a grid representation.

Learning phase – 500 random samples are generated in the occupancy map and connected to neighbouring nodes if obstacle-free. These are stored in `Samples`.

Query phase – The samples are copied, start and goal states are temporarily added and connected to neighbours, then A* computes the shortest path.

Finally, grid coordinates (x,y) are converted into world coordinates for the control stage.

4 Control

- Start simulation – `cd ~/robot_assignment_ws, ./startWorld`
- Save navigation node – `nano ../navigator_node.py` (copy code from repository)
- Give permissions – `chmod +x ../navigator_node.py`
- Create launch file – `nano ../nav.launch` (copy code from repository)
- Run navigation – `roslaunch surveillance_bot nav.launch`